

AI_USAGE.md

AI-Assisted Development Report — LLD Practice Platform

1. Purpose

AI was used as a development assistant throughout the LLD Practice Platform assignment. It was used selectively for research, brainstorming, debugging, validation, reviewing, and documentation. The core product direction, architecture, implementation, and engineering trade-offs remained the developer's responsibility.

The goal of using AI was to accelerate iteration and challenge design decisions, not to delegate ownership of the application.

2. Meaningful AI-Assisted Decisions

Decision 1 — Evaluation Rubric Design

What AI suggested: A structured rubric would make LLD feedback more consistent and useful than a single generic score.

What was accepted: An eight-criterion rubric covering Requirement Understanding, Class Responsibilities, Coupling & Cohesion, Encapsulation & Interfaces, Abstraction & Patterns, Extensibility, Edge Cases & Testability, and Explanation Quality.

Why: These criteria directly support the assignment's requirement for explainable feedback while allowing multiple valid LLD solutions.

Decision 2 — Evaluator Abstraction

What AI suggested: Separate the evaluator contract from the concrete evaluation implementation.

What was accepted: An **Evaluator** abstraction with a concrete Gemini-based evaluator.

Why: The practice flow should not depend permanently on one evaluation mechanism. This also supports future rule-based or human evaluation.

Decision 3 — Structured LLM Evaluation

What AI suggested: Ask the model for structured output instead of parsing free-form feedback.

What was accepted: The evaluator returns criterion-level score, evidence, concern, suggestion, and confidence, along with overall feedback, strengths, and improvements.

Why: Structured output is easier to validate, persist, display, and evolve.

3. Additional AI-Assisted Decisions

Decision 4 — Deterministic Validation + LLM Evaluation

What AI suggested: Do not rely entirely on the LLM for basic validity checks.

What was accepted: Required submission sections are validated deterministically before qualitative LLM evaluation. The generated evaluation response is also validated before persistence.

Why: Basic application rules should be predictable, while subjective LLD judgment is where an LLM provides value.

Decision 5 — Evaluation Failure Handling

What AI suggested: Treat evaluation as a stateful operation and preserve the learner's submission when an external model fails.

What was accepted: The attempt moves through **Submitted** → **Evaluating** → **Completed / Failed**. The submission is persisted before evaluation, and a failed evaluation can be retried without submitting the work again.

Why: External AI services can be slow or temporarily unavailable. Learner work should not be lost, and failure should be recoverable.

4. AI-Assisted Coding & Debugging

AI was also used during implementation to help reason about code structure, diagnose errors, review API behavior, and refine documentation. This included debugging backend integration issues and checking validation/evaluation flows.

The development process remained iterative: AI suggestions were inspected against the actual codebase, tested locally, and changed when they did not fit the project's scope or requirements.

Examples of development assistance included:

- Reasoning about controller/service responsibilities and keeping evaluation logic separated.
- Reviewing validation rules and structured evaluation response handling.
- Helping diagnose external Gemini API/model availability issues and integration errors.
- Reviewing failure and retry behavior after an evaluation service error.
- Improving README, design documentation, and explanation of engineering trade-offs.

5. What Was Accepted vs. Rejected

AI direction	Decision	Reason
Structured rubric	Accepted	Improves consistency and explainability.
Evaluator abstraction	Accepted	Supports future evaluator implementations.
Structured JSON output	Accepted	Reliable validation and UI rendering.
Deterministic validation before LLM	Accepted	Keeps objective rules predictable.
Stateful failure + retry	Accepted	Protects learner work and improves reliability.
Unnecessary distributed infrastructure	Rejected	Outside MVP scope and adds complexity without enough value.

6. What I Did Not Delegate to AI

AI did not independently define or own the final application. The following areas were deliberately kept under developer control:

- Final product scope and the learner practice flow.
- Final domain model and MongoDB schema decisions.
- API structure, controller behavior, and persistence flow.
- Frontend page flow and implementation.
- Gemini integration and handling of real API failures.
- Test creation, execution, and interpretation of test results.
- Final architecture, trade-offs, limitations, and extensibility decisions.
- Final review of generated code and documentation before inclusion.
-

7. Engineering Ownership

AI suggestions were treated as proposals rather than authoritative answers. The developer evaluated whether each suggestion was appropriate for the assignment, implemented selected ideas, tested them, and rejected ideas that introduced unnecessary complexity.

This was especially important because the assignment explicitly values engineering judgment. For example, the project remains a modular monolith rather than introducing Kafka, Redis, Kubernetes, microservices, or other distributed infrastructure that was not necessary for the MVP.

8. Result of AI-Assisted Development

- A fixed and explainable evaluation rubric was established.
- Evaluation became replaceable through an evaluator abstraction.
- LLM output became structured and validated before persistence.
- Learner submissions are protected from temporary evaluator failures.
- The platform supports retrying failed evaluations.
- The documentation clearly separates product/domain decisions from AI-assisted suggestions.
-

9. Summary

AI accelerated research, brainstorming, debugging, validation, and documentation, but the final solution was reviewed and owned by the developer. The most valuable use of AI was not generating large amounts of code; it was helping challenge design decisions and identify practical reliability and extensibility patterns.

The resulting implementation intentionally balances AI capability with deterministic application behavior: objective validation remains predictable, while the LLM is used where qualitative LLD judgment is useful.

Developer: Naman Jain

Project: LLD Practice Platform — Engineering Assignment